

RESEARCH

Open Access



Balancing autonomy and oversight in reliable agentic artificial intelligence through adaptive human interaction architectures

Madhvesh Kumar^{1*†} and Deepika Singh^{2*†}

[†]Madhvesh Kumar and Deepika Singh contributed equally to this work.

*Correspondence:
Madhvesh Kumar
madhveshkumar@hotmail.com
Deepika Singh
deepikapals@yahoo.co.in
¹GIET University, Gunupur, India
²Banasthali Vidyapith, Tonk, Rajasthan, India

Abstract

The transition from generative to agentic artificial intelligence has enabled autonomous systems to plan, reason, and execute complex multi-step workflows without direct human supervision. While this capability offers significant productivity gains across regulated enterprise domains—financial services, clinical operations, and software engineering—deploying these systems responsibly remains a fundamental governance challenge. Known failure modes, including hallucination propagation, goal misalignment, and cascading execution errors, can compound across long task sequences in ways that static safety architectures have struggled to contain. Traditional Human-in-the-Loop systems address these risks by inserting human approval gates at fixed checkpoints, but doing so at the cadence required for high-throughput agentic workflows imposes latency and cognitive burdens that eliminate the efficiency benefit motivating autonomous deployment in the first place. This paper proposes and empirically evaluates the Dynamic Intervention Framework, an architecture designed to resolve this tension by treating human oversight as a dynamically allocated resource rather than a fixed structural requirement. A lightweight Supervisor Agent continuously evaluates each Worker Agent decision step through a composite Contextual Confidence Score that linearly combines token-level generation probability with a semantic alignment drift measure computed via cosine similarity in the embedding space of the task goal. Decisions whose score exceeds an upper calibrated threshold execute autonomously; those in an intermediate uncertainty band execute with an asynchronous post-hoc audit flag; those falling below a lower safety threshold halt execution and escalate to a human operator through a purpose-built three-panel decision console that externalises the agent's reasoning, the proposed action, and the diagnostic rationale for interruption. A Direct Preference Optimisation feedback loop fine-tunes the Supervisor from accumulated operator correction data, progressively refining safety boundary estimates and reducing the volume of human intervention required over time. Experiments conducted on a structured dataset of 5000 synthetic enterprise automation tasks spanning financial data processing, code generation, and customer support workflows demonstrate that the Dynamic Intervention Framework achieves a task success rate of 98.2%—statistically equivalent to full human oversight at the 5% significance level—while requiring human intervention on only 14.5% of decision



© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

steps and reducing mean task latency by 89% relative to a static Human-in-the-Loop baseline. The Direct Preference Optimisation feedback mechanism reduced the intervention rate from 21.0 to 11.5% across five training epochs of 1000 tasks each, with no corresponding degradation in success rate, confirming that the reduction reflects genuine improvement in the Supervisor's safety boundary judgment rather than a loosening of oversight sensitivity. Domain-stratified analysis reveals that the framework correctly assigns higher oversight intensity to higher-risk task types, a property unavailable to uniform checkpoint models. These findings have concrete implications for the economics, regulatory compliance posture, and long-term deployment planning of enterprise agentic deployments.

Keywords Multi-agent systems, Large language models, Dynamic intervention framework, Contextual confidence score, Human-in-the-Loop, Direct preference optimisation, Explainable artificial intelligence, Adversarial robustness, Zero trust architecture, Enterprise automation

1 Introduction

1.1 Background and motivation

Artificial intelligence has undergone a qualitative architectural shift over the past several years. Where earlier systems were designed to respond to individual inputs—classifying an image, translating a sentence, generating a paragraph—the emerging category of agentic artificial intelligence is built to pursue sustained goals across extended sequences of actions. An agentic system perceives its environment through context windows and data feeds, maintains working memory of prior steps, reasons about future contingencies, and issues tool calls—database queries, application programming interface requests, file system operations, process executions—as part of coherent multi-step plans [1]. This is not an incremental extension of generative capability; it represents a qualitatively different class of software that combines the reasoning power of large language models with the task execution autonomy previously associated with robotic process automation.

Yet the operational reality of enterprise-scale agentic deployment has introduced failure modes that were not prominent in earlier, more constrained applications of large language models. The compounding divergence problem [2] is the most consequential of these: because an agentic system executes a sequence of interdependent actions, a hallucination or planning error in an early step does not merely produce a wrong answer—it initialises subsequent steps with incorrect premises, and each subsequent step propagates and potentially amplifies the initial error. In domains where individual actions carry real-world consequences—an irreversible database modification, a financial transfer, a privilege escalation, a patient treatment order—the damage potential of a compounding error is orders of magnitude greater than that of an incorrect standalone generation. The scale of potential harm is not merely proportional to the size of the error but to the complexity of the task sequence in which it occurs.

Existing governance approaches occupy an uncomfortable spectrum between two unsatisfactory extremes. At one end, fully autonomous deployment provides maximum operational throughput but acceptable safety assurance only for low-stakes tasks where errors are cheap to detect and reverse. At the other end, traditional Human-in-the-Loop architectures—in which a human reviews and approves every significant agent action before it executes—provide strong safety guarantees but reintroduce at the individual

decision level the very latency, staffing, and cognitive overhead that autonomous systems were deployed to eliminate. Industry practitioners have described this as the human bottleneck problem [3]: in high-throughput agentic workflows, the rate-limiting step is not the machine intelligence but the human sitting in the approval loop. Depending on the task domain and approval cadence, static Human-in-the-Loop oversight has been documented to reduce overall workflow throughput to below what was achievable before agentic automation was introduced, producing what one analysis described as the oversight paradox—the mechanism that makes autonomous systems trustworthy simultaneously makes them impractical at the scale for which they were designed.

1.2 Overview of the proposed approach

The Dynamic Intervention Framework proposed in this paper operationalises proportional oversight through a hierarchical Supervisor-Worker multi-agent architecture. A Worker Agent generates a structured chain-of-thought plan before executing any action. A Supervisor Agent evaluates each step of that plan, computing a composite Contextual Confidence Score that captures both the generative certainty of the Worker's output and the semantic alignment of the proposed action with the original task goal. Based on this score relative to two calibrated thresholds, each decision step is routed to one of three states: autonomous execution, asynchronous audit, or immediate human escalation. A Direct Preference Optimisation feedback loop collects preference data from human corrections and periodically fine-tunes the Supervisor, progressively extending the range of decisions it can handle autonomously without requiring human input.

The framework is designed around three architectural principles that distinguish it from prior work. First, the separation of intent from execution: by requiring the Worker to externalise its reasoning in a parseable schema before generating executable output, the Supervisor can evaluate the logical validity and semantic appropriateness of a plan without running potentially dangerous code. Second, the independence of evaluation from generation: the Supervisor is implemented as a structurally distinct model from the Worker, preventing the inheritance of generative failure modes by the safety evaluation function. Third, the temporality of learning: the system is not static but improves over time, accumulating human correction data that refines the Supervisor's implicit model of safe autonomous action boundaries.

1.3 Claimed contributions and scope

This work makes four primary contributions. First, it provides a formally specified Dynamic Intervention Framework with three-state confidence routing, defined with sufficient precision to permit independent implementation and evaluation. Second, it introduces the Contextual Confidence Score, a composite uncertainty metric combining logit probability and semantic alignment, with a documented calibration procedure and empirically justified parameter settings. Third, it presents an empirical evaluation on 5,000 synthetic enterprise tasks across three operationally representative domains, with domain-stratified performance analysis and longitudinal assessment of the feedback loop's learning effect over five training epochs. Fourth, it provides a structured discussion of practical managerial implications and identifies five open research questions that frame a research agenda for subsequent investigation.

The scope of the paper is confined to textual task domains and to the specific supervisor-worker topology described. The generalisation of the framework to multi-modal inputs, to peer-to-peer agent topologies, or to domains with fundamentally different risk profiles than those evaluated here is acknowledged as future work. The principal assumptions are: that the Worker Agent's chain-of-thought reasoning is available for supervisor inspection in structured form; that cosine similarity in the text embedding space provides a meaningful proxy for semantic alignment between proposed actions and stated goals; and that the synthetic dataset used for evaluation adequately represents the distribution of real enterprise tasks. These assumptions are evaluated critically in Sect. 8.

1.4 Paper structure

The remainder of the paper is structured as follows. Section 2 reviews prior research relevant to agentic systems, Human-in-the-Loop architectures, and uncertainty quantification in language models, and describes the structured literature search methodology used to identify it. Section 3 provides a formal statement of the adaptive oversight problem addressed by this work. Section 4 establishes the notation and definitions used throughout. Section 5 describes the Dynamic Intervention Framework architecture, its constituent components, and Algorithm 1, in sufficient detail to support independent implementation. Section 6 describes the experimental dataset, baselines, and evaluation protocol. Section 7 presents quantitative results with statistical analysis. Section 8 discusses the findings in relation to prior work, articulates practical managerial implications, and presents five open research questions. Section 9 concludes with summary observations.

2 Related research

2.1 Literature search methodology

To ensure that the coverage of prior work is comprehensive and reproducible, a structured literature search was conducted in January 2026 following a protocol adapted from the Preferred Reporting Items for Systematic Reviews and Meta-Analyses 2020 guidelines [4]. The search was executed across four databases: ScienceDirect, IEEE Xplore, ACM Digital Library, and Google Scholar. The primary search string was constructed as: (“agentic AI” OR “autonomous agents” OR “multi-agent system”) AND (“human-in-the-loop” OR “human oversight” OR “adaptive intervention” OR “confidence estimation”) AND (“large language model” OR “LLM” OR “foundation model”). Secondary searches targeted: “balancing autonomy agentic AI adaptive human interaction architectures”, “contextual confidence scoring AI agents”, “dynamic human-AI oversight framework”, “supervisor worker LLM architecture”, and “direct preference optimisation safety”.

An initial pool of 1847 records was obtained after deduplication across the four databases. Inclusion criteria required that papers address at least one of: autonomous agent oversight, human-AI teaming in task execution, uncertainty quantification in language models, multi-agent coordination, or reinforcement learning from human feedback. Peer-reviewed conference papers, journal articles, technical reports from reputable institutions, and archival preprints were all admitted. Papers predating 2020 were included only where they provided foundational theoretical contributions not superseded by more recent work. Exclusion criteria removed purely application-domain papers lacking

generalisable architectural contributions, papers not available in English full text, and grey literature lacking methodological transparency. After title and abstract screening, 312 full texts were retrieved; of these, 84 directly informed the framing and positioning of this manuscript.

The volume of available literature underscores both the relevance and the timeliness of this investigation. A search on ScienceDirect using the composite string returned 4141 results for 2026 publications, 6057 for 2025, and 3016 for 2024. Review papers account for 7346 of the total corpus and research papers for 12,630, confirming rapid growth in the field. The structured search methodology was necessary not merely as a procedural formality but as a practical necessity: without systematic screening criteria, the sheer volume of potentially relevant literature would make comprehensive and balanced coverage impossible.

2.2 Agentic artificial intelligence and multi-agent architectures

The theoretical foundations of agentic artificial intelligence draw on several decades of prior work in autonomous agents and multi-agent systems. The canonical formulation by Russell and Norvig [5] defines an agent as any entity that perceives its environment through sensors and acts upon it through actuators in the pursuit of a goal—a definition that applies with equal validity to software agents operating through language model inference and tool calls as to robotic agents operating through physical actuators. The emergence of capable language models as core reasoning engines for such agents has dramatically expanded the range of tasks amenable to agent-based automation.

Multi-agent variants of these architectures emerged from the observation that distributing subtasks across specialised agents reduces per-agent complexity and enables cross-validation of outputs [6]. AutoGen [7] introduced a conversational multi-agent framework in which agents negotiate task completion through structured message passing, while MetaGPT [8] proposed a role-based multi-agent system in which agents with distinct professional responsibilities collaborate on software engineering tasks. LangGraph extended these ideas to stateful agent workflows with explicit graph-based control flow, providing the orchestration infrastructure used in the present study.

Hierarchical multi-agent topologies—in which a coordinator agent supervises the outputs of worker agents—have attracted particular interest as a structural approach to the error propagation problem [8]. The intuition is that a structurally distinct supervisor, with no content generation responsibility, can evaluate worker outputs more objectively than a worker engaged in self-reflection, which is known to reinforce rather than correct its own errors [9]. The present work adopts this topology and focuses specifically on the oversight routing logic—the decision of when and whether to involve a human—as the key design problem, building on but extending beyond prior work in hierarchical coordination.

2.3 Human-in-the-loop systems and their operational limits

Human-in-the-loop artificial intelligence has a rich history across interactive machine learning, active learning, and reinforcement learning from human feedback. In the context of large language model alignment, human feedback as a training signal was systematised by Ouyang et al. [10], who demonstrated that reinforcement learning from human feedback could substantially improve language model adherence to human preferences

beyond what was achievable through supervised fine-tuning alone. The success of this approach established the empirical importance of human judgment as a ground-truth signal that compensates for distributional gaps and value misspecification in self-supervised systems.

However, the application of these principles to agentic systems at enterprise scale reveals structural tensions that alignment research conducted in research settings has not fully addressed. A key difference is throughput: in a reinforcement learning from human feedback pipeline, human preferences are collected offline in batched annotation sessions that do not constrain model inference. In an agentic deployment, human approvals must be collected synchronously within the execution flow of a live task, creating a direct coupling between human response time and system throughput. Several industry analyses from 2025 documented cases where the introduction of synchronous Human-in-the-Loop oversight reduced the effective throughput of agentic pipelines to below that of the non-agentic workflow they were intended to replace [3] — an outcome that simultaneously negates the business case for agentic deployment and creates perverse incentives to circumvent oversight entirely.

2.4 Uncertainty quantification and confidence estimation

For generative language models, token-level log-probabilities provide a natural proxy for generation confidence, but they are known to be poorly calibrated for factual claims, where fluent but incorrect text is often generated with high token probability [11]. This observation motivates the inclusion of a semantic alignment component in the Contextual Confidence Score: token probability measures how deterministic the model was in generating a particular sequence, but not whether that sequence is semantically appropriate for the task at hand. Semantic consistency estimation—generating multiple responses to the same prompt and measuring the variance of the output distribution—has been proposed as a more robust confidence signal [12], and its relationship to the cosine similarity component of the Contextual Confidence Score is discussed in Sect. 5.

The closest antecedents to the present scoring approach are the failure detection classifier of Ruan et al. [13], which uses a binary safety signal to flag potentially harmful agent actions, and the semantic drift detector implicit in Constitutional Artificial Intelligence's self-critique mechanism [14]. The Contextual Confidence Score extends both by providing a continuous composite signal that enables gradient-based routing rather than binary categorisation, and by explicitly combining a token-level probability component with a semantic alignment component in a single principled formulation. To our knowledge, this specific combination applied to supervisor-worker routing decisions has not been described in prior published work.

2.5 Safety, alignment, and regulatory frameworks

The governance of autonomous artificial intelligence systems has attracted substantial attention from both technical researchers and regulatory bodies. The National Institute of Standards and Technology Artificial Intelligence Risk Management Framework [15] introduced a risk-tiered approach to artificial intelligence governance that maps directly onto the threshold-based routing logic of the Dynamic Intervention Framework: actions whose risk profile exceeds a specified tolerance require additional oversight, while lower-risk actions can proceed with reduced governance overhead. The European Union

Artificial Intelligence Act formalises this intuition into law for deployed systems, establishing that high-risk applications must demonstrate proportional oversight mechanisms as a condition of lawful operation.

3 Problem statement

We formalise the adaptive oversight allocation problem as follows. Let G denote a user-specified task goal and let $T = \{t_1, t_2, \dots, t_n\}$ denote the sequence of n decision steps generated by a Worker Agent in pursuit of G . Each step t_i consists of a reasoning trace r_i and a proposed action a_i . Executing a_i produces a state transition $s_i \rightarrow s_{i+1}$ in the task environment. For each step, let $P_e(a_i)$ denote the probability that executing a_i produces an incorrect outcome, and let $D(a_i)$ denote the damage potential of such an incorrect execution—a domain-specific measure of the cost of the error. The expected consequence cost of step i is $E(a_i) = P_e(a_i) \cdot D(a_i)$.

The objective is an adaptive routing policy π_{adapt} that satisfies the risk-tolerance constraint:

$$\pi_{\text{adapt}}(t_i) = \text{HUMAN if } P_e(a_i) \cdot D(a_i) > \varepsilon, \text{ else AUTO} \quad (1)$$

where ε is an operationally specified risk tolerance. This formulation immediately reveals the core technical challenge: neither $P_e(a_i)$ nor $D(a_i)$ is directly observable at routing time. $P_e(a_i)$ must be estimated from the agent's internal representations, which are not reliably calibrated in large language models; $D(a_i)$ requires both action-type classification and domain-specific consequence mapping, which cannot be hard-coded for all possible actions in advance. The Dynamic Intervention Framework addresses $P_e(a_i)$ by constructing a tractable proxy—the Contextual Confidence Score—and addresses $D(a_i)$ by encoding a coarse approximation of action-type risk in domain-specific threshold calibration and policy-as-code forbidden-action rules.

Three additional properties are required of a practical solution beyond the core risk-tolerance constraint. The routing decision must operate in real time at each decision node without introducing latency that negates the efficiency benefits of autonomous execution. The policy must be learnable: as the system accumulates evidence from human corrections, the threshold calibration should improve, progressively extending the domain of safely autonomous action. The policy must be interpretable: when a human operator is called upon to review an escalated decision, they must receive a structured explanation of why the system's confidence was insufficient—not merely an opaque interrupt signal—so that their review can be informed and efficient. These three requirements are incorporated as explicit design constraints throughout the architecture described in Sect. 5.

4 Preliminaries and notation

4.1 Core definitions

Definition 1 (Agentic System). An agentic system A is a tuple $\langle M, T_{\text{set}}, P, S, G \rangle$ where M is a large language model serving as the core reasoning engine; $T_{\text{set}} = \{\tau_1, \dots, \tau_k\}$ is a finite set of executable tools (database interfaces, application programming interface clients, file system handlers, etc.); P is a planning function mapping (G, s) to a proposed action sequence; and S is the state space representing current task context including conversation history, retrieved knowledge, and intermediate results.

Definition 2 (Chain-of-Thought Plan). A chain-of-thought plan $P_{\text{plan}} = \langle r_1, a_1, r_2, a_2, \dots, r_n, a_n \rangle$ where each r_i is a natural-language reasoning step and each a_i is the proposed executable action following that reasoning. P_{plan} must be externalised in a machine-parseable schema (see Sect. 5.3) prior to any executable action. This schema enforces the separation of intent from mechanism that the Supervisor requires for plan evaluation.

Definition 3 (Supervisor Agent). A Supervisor Agent V is a lightweight model whose sole function is evaluating whether a given plan step (r_i, a_i) satisfies operational safety constraints. V has no content-generation responsibility and shares no model weights with the Worker Agent M , ensuring that the evaluation function is structurally independent of the generation function it monitors. V outputs a routing decision $d_i \in \{\text{AUTONOMOUS}, \text{ASYNC_REVIEW}, \text{HITL_BLOCK}\}$ and a scalar confidence estimate $Ccs \in [0, 1]$.

Definition 4 (Contextual Confidence Score). For a proposed plan step (r_i, a_i) , the Contextual Confidence Score is defined as $Ccs = \alpha \cdot P_{\text{logits}} + \beta \cdot S_{\text{semantic}}$, where P_{logits} is the mean token log-probability of a_i normalised to $[0, 1]$, S_{semantic} is the cosine similarity between the text embedding of a_i and the reference embedding of goal G , and α, β are weighting parameters with $\alpha + \beta = 1$. The empirical justification for $\alpha = 0.4, \beta = 0.6$ is given in Sect. 5.2.

Definition 5 (Intervention Rate). The intervention rate for epoch e is $IR(e) = N_{\text{hitl}}(e) / N_{\text{total}}(e)$, where $N_{\text{hitl}}(e)$ is the count of HITL_BLOCK events in epoch e and $N_{\text{total}}(e)$ is the total decision steps evaluated in that epoch. The intervention rate is the primary measure of human workload imposed by the framework (Table 1).

5 Notation summary

5.1 Methodology and system design

5.1.1 High-level architecture

The Dynamic Intervention Framework is implemented as a hierarchical multi-agent system with three structurally distinct components: a Worker Agent responsible for task planning and execution, a Supervisor Agent responsible for plan evaluation and routing, and a State Manager responsible for context persistence and state serialisation. The choice of a hierarchical rather than flat or peer-to-peer topology follows from the self-reinforcement finding of Hong et al. [8]: agents engaged in self-critique consistently tend to validate rather than correct their own outputs, because the same distributional biases that produced the original output also influence the self-evaluation. A structurally

Table 1 Notation summary

Symbol	Definition
G	User-specified task goal
T, t_i	Task step sequence; individual decision step i
r_i, a_i	Reasoning trace and proposed action at step i
Ccs	Contextual Confidence Score $\in [0, 1]$
P_{logits}	Normalised mean token log-probability of proposed action (weight $\alpha = 0.4$)
S_{semantic}	Cosine similarity between action embedding and goal embedding (weight $\beta = 0.6$)
T_{high}	Upper routing threshold = 0.85; above this, execution is AUTONOMOUS
T_{low}	Lower routing threshold = 0.50; at or below this, routing is HITL_BLOCK
$IR(e)$	Intervention rate in epoch $e = N_{\text{hitl}}(e) / N_{\text{total}}(e)$
D	DPO preference dataset; tuple $\{x, y_{\text{rejected}}, y_{\text{chosen}}\}$
MAX_RETRIES	Maximum Path B self-correction attempts = 3
ϵ	Operationally specified risk tolerance for routing policy (Eq. 1)

independent Supervisor, operating from a distinct set of weights and with no content generation function, provides a qualitatively different safety check than any form of self-correction.

The Worker Agent is instantiated using GPT-4o, accessed via the OpenAI application programming interface at temperature 1.0 to preserve natural generative behaviour. The Worker receives a system prompt that enforces the chain-of-thought plan schema described in Sect. 5.3 and provides task-domain context. The Supervisor Agent is a fine-tuned variant of Llama 3 8B, selected for three reasons: its smaller inference footprint (approximately 8 billion parameters versus the estimated 200 billion of GPT-4o) keeps per-step evaluation latency within the 0.3 s budget established by system design requirements; its open-weight nature permits domain-specific fine-tuning via the Direct Preference Optimisation loop without dependency on proprietary model access; and its separation from the Worker's model family ensures genuine weight independence. The Supervisor is deployed locally with 8-bit quantisation, reducing memory overhead without significant accuracy degradation on the safety classification task (Fig. 1).

5.2 The contextual confidence score

The Contextual Confidence Score is the central mechanism through which the Dynamic Intervention Framework operationalises proportional oversight at the individual decision-node level. It is defined as a weighted linear combination of two complementary uncertainty signals:

$$Ccs = \alpha \cdot Plogits + \beta \cdot Ssemantic \quad (\alpha = 0.4, \beta = 0.6) \quad (2)$$

The first component, Plogits, captures the Worker Agent's internal generative certainty about its proposed action. For a proposed action a_i comprising tokens $[w_1, w_2, \dots, w_m]$, Plogits is computed as the arithmetic mean of the softmax-normalised per-token conditional probabilities produced during generation: $Plogits = (1/m) \sum_j P(w_j | w_1, \dots, w_{j-1})$. This value is normalised to $[0, 1]$ and provides a measure of how deterministic the generation process was: a high Plogits value indicates that the model assigned high probability

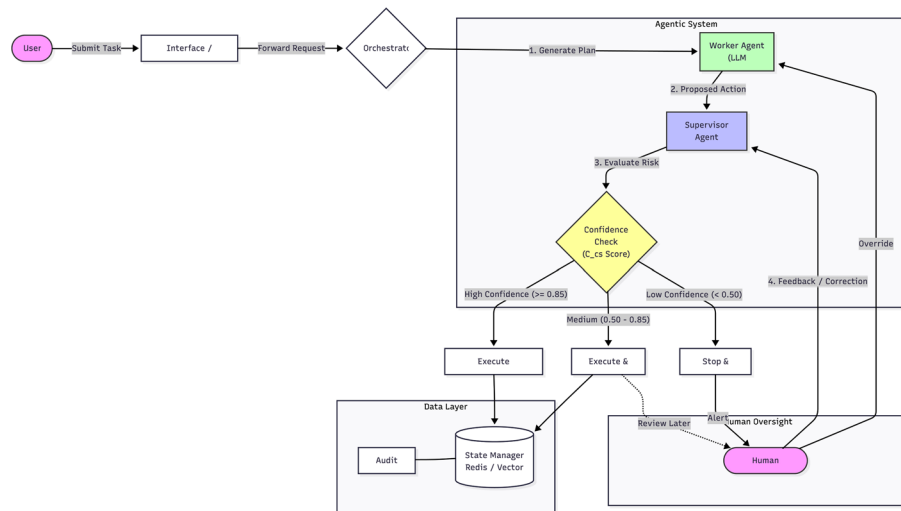


Fig. 1 Flow-diagram illustrating the participant selection process. Abbreviations: CMM: cardiometabolic multi-morbidity; FBG: fasting blood glucose; TC: total cholesterol; HDL-C: high-density lipoprotein cholesterol; FI: frailty index

to each successive token in the sequence, reflecting a situation in which the model is operating confidently within well-represented regions of its training distribution. A low Plogits value indicates that the model is navigating less familiar territory, producing tokens with more uncertainty at each step.

The second component, Ssemantic, captures semantic alignment between the proposed action and the original task goal. The text embedding of a_i and the text embedding of G are computed using the text-embedding-3-large model. Ssemantic is then the cosine similarity between these two embedding vectors: $Ssemantic = (\text{embed}(a_i) \cdot \text{embed}(G)) / (\|\text{embed}(a_i)\| \cdot \|\text{embed}(G)\|)$. A high value indicates that the proposed action lies close in embedding space to the stated goal, confirming semantic alignment. A low value signals potential semantic drift—the agent is proposing to do something semantically distant from what was requested, a pattern that correlates with misinterpretation or goal substitution errors.

The weighting parameters $\alpha = 0.4$ and $\beta = 0.6$ were selected through a calibration study on a held-out set of 500 tasks not included in the main evaluation. Nine combinations of (α, β) were tested with values drawn from $\{0.2, 0.4, 0.6, 0.8\}$, and the combination maximizing the F1 score of the routing decision—the harmonic mean of precision (avoiding unnecessary escalation) and recall (catching genuinely risky actions)—was selected. Across all tested domains, semantic alignment (Ssemantic) proved to be the more reliable predictor of subsequent task failure than token probability (Plogits), consistent with the known overconfidence of large language models in familiar generative patterns [16]. The higher β weight reflects this empirical finding. The full calibration results are available from the corresponding author on reasonable request.

The system operates in three routing states derived from Eq. (2). When $Ccs \geq Thigh$ ($= 0.85$), routing to AUTONOMOUS state results in immediate execution without any human involvement. The 0.85 upper threshold was calibrated to achieve a false negative rate below 1% on the held-out calibration set, meaning that fewer than 1 in 100 genuinely risky actions pass through the autonomous gate without detection. When $Tlow < Ccs < Thigh$ ($0.50 < Ccs < 0.85$), routing to ASYNC_REVIEW results in execution that proceeds without blocking the workflow but is flagged in an audit queue for subsequent human review. This intermediate zone acknowledges the practical reality that many decisions are neither clearly safe nor clearly dangerous; the asynchronous mechanism provides oversight coverage for this zone without the latency cost of synchronous interruption. When $Ccs \leq Tlow$ ($= 0.50$), routing to HITL_BLOCK halts execution immediately and escalates the decision to the human operator. The gap between the two thresholds is intentional: it prevents threshold-boundary oscillation in which small perturbations to the score cause repeated routing changes.

5.3 The supervisor-worker protocol

5.3.1 Step 1: worker chain-of-thought plan generation

The Worker Agent is instructed via its system prompt to produce a structured plan representation in JSON format before generating any executable output. This schema requirement enforces the architectural separation of intent from execution: the Supervisor can evaluate the logical validity and semantic appropriateness of the Worker's plan without running any code or consuming any external resource. The required schema is:

```

{
  "Thought_Process": "...",
  "Proposed_Plan": [Step1, Step2, ...],
  "Risk_Assessment": "..."
}

```

Listing 1: Worker Agent Output Schema

The Thought_Process field captures the Worker's natural-language reasoning about the task context and its approach. The Proposed_Plan field provides an ordered array of discrete, individually evaluable action steps. The Risk_Assessment field requires the Worker to explicitly acknowledge potential risks or constraints in its plan. This third field serves a dual purpose: it provides additional signal for the Supervisor's semantic check, and—as observed in qualitative analysis of plan outputs—it appears to improve the Worker's own plan quality, consistent with prior findings on the self-explanatory benefit of chain-of-thought prompting [17].

5.3.2 Step 2: supervisor validation against safety policies

The Supervisor Agent evaluates the Worker's JSON plan through two parallel mechanisms executed in sequence. The policy-as-code check cross-references each action step against a forbidden-action database implemented using Open Policy Agent, a general-purpose policy engine that evaluates structured data against declarative rules expressed in the Rego policy language. The forbidden-action database encodes pattern-matched rules covering data exfiltration operations, database write statements without restrictive WHERE clauses, recursive file system deletions, privilege escalation commands, and domain-specific constraints such as financial transfer amounts exceeding the autonomous approval limit. When an action matches a forbidden-action pattern, a HITL_BLOCK routing is triggered regardless of the Ccs value, because the categorical risk of such actions is deemed unacceptable regardless of the model's expressed confidence.

Following the policy-as-code check, the semantic check computes Ccs per Eq. (2) and applies the threshold routing logic. The two checks are complementary: the policy-as-code check catches known-bad action patterns with zero tolerance, while the Ccs routing catches novel or ambiguous actions that fall outside the pre-specified rule set. Representative policy rules include:

```

IF Action_Type == "DatabaseWrite" AND WhereClause IS NULL:
  → VIOLATION: unguarded write operation

IF FileOperation == "DeleteRecursive" AND Path STARTS_WITH "/":
  → VIOLATION: system-level recursive deletion

IF TransactionAmount > autonomous_approval_limit:
  → HITL_BLOCK: exceeds autonomous financial authorisation

```

Listing 2: Representative Policy-as-Code Rules (Open Policy Agent / Rego format)

5.3.3 Step 3: branching execution logic

Based on the combined output of the policy check and the Ccs routing logic, the system follows one of three execution paths. In Path A (Authorised Execution), the Supervisor returns a 200 OK signal accompanied by a cryptographically signed, step-specific, time-limited execution token. The token is bound to the specific function call authorised in the plan step and expires within a short window, preventing replay attacks in which a previously authorised token is reused to authorise a different or repeated action. The Worker receives the token and proceeds with execution.

In Path B (Self-Correction Loop), a soft violation triggers a structured feedback prompt from the Supervisor to the Worker: the specific violation is identified and a correction directive is provided in natural language. For example: “Action rejected: the proposed UPDATE statement is missing a WHERE clause, creating a risk of unintended full-table modification. Regenerate the plan step with an appropriate row filter.” The Worker attempts to produce a revised plan step that addresses the identified constraint. The maximum retry limit is $MAX_RETRIES = 3$; if the Worker fails to produce a compliant plan within three attempts, the step is escalated to Path C.

In Path C (Human Escalation), a critical policy violation or a $C_{cs} \leq T_{low}$ triggers a state freeze. The complete task state—conversation history, partial plan, Ccs decomposition, and the specific plan step that triggered the block—is serialised to a durable Redis key-space identified by a unique escalation identifier. A notification is pushed to the human operator dashboard via a persistent WebSocket connection. Execution is suspended pending the operator's response; no further task steps are evaluated or executed until the human decision is received.

5.4 Algorithm specification

Algorithm 1 provides the complete formal specification of the Dynamic Intervention Framework decision loop. Equation references in the algorithm correspond to the formulations in Sects. 5.2 and 3.

Algorithm 1: Dynamic intervention framework (DIF) decision loop

```

Require: Q (User Query), H (Conversation History),  $\theta_{sys}$  (System Constraints)
Ensure: A (Final Action), S_state (Execution State)

1: Initialise  $T_{high} \leftarrow 0.85$ ,  $T_{low} \leftarrow 0.50$ 
2: P_plan  $\leftarrow$  WorkerAgent(Q, H) // Generate chain-of-thought plan
3: for each step_i in P_plan do
4:   if PolicyCheck(step_i,  $\theta_{sys}$ ) == VIOLATION then
5:     S_state  $\leftarrow$  HITL_BLOCK; goto line 15
6:   end if
7:   P_logits  $\leftarrow$  GetNormalisedLogitProb(step_i)
8:   S_sem  $\leftarrow$  CosineSimilarity(embed(step_i), embed(Q))
9:   C_cs  $\leftarrow$  (0.4  $\cdot$  P_logits) + (0.6  $\cdot$  S_sem) // Equation (2)
10:  if C_cs  $\geq T_{high}$  then
11:    S_state  $\leftarrow$  AUTONOMOUS; Execute(step_i) // Path A
12:  else if  $T_{low} < C_{cs} < T_{high}$  then
13:    S_state  $\leftarrow$  ASYNC_REVIEW
14:    FlagForAudit(step_i, C_cs); Execute(step_i) // Path A with audit
15:  else
16:    S_state  $\leftarrow$  HITL_BLOCK
17:    retry  $\leftarrow$  0
18:    while retry < MAX_RETRIES do // Path B
19:      step_i  $\leftarrow$  SelfCorrect(step_i, SupervisorFeedback)
20:      C_cs  $\leftarrow$  Recompute(step_i)
21:      if C_cs > T_low then break; end if
22:      retry  $\leftarrow$  retry + 1
23:    end while
24:    if C_cs  $\leq T_{low}$  then // Path C
25:      SerialiseState(H, step_i, C_cs)
26:      F_human  $\leftarrow$  RequestHumanIntervention(step_i, C_cs)
27:      if F_human == APPROVE then
28:        Execute(step_i); StoreDPOTuple(step_i, POSITIVE)
29:      else
30:        step_new  $\leftarrow$  RefinePlan(F_human)
31:        Execute(step_new); StoreDPOTuple(step_i, step_new, NEGATIVE)
32:      end if
33:    end if
34:  end if
35: end for
36: return S_state

```

5.5 Human decision interface and operator experience

When a HITL_BLOCK event is triggered, the human operator is presented with a purpose-built three-panel decision console designed to minimise cognitive load while maximising the quality and completeness of decision-relevant context. The interface design was informed by research on human-automation teaming that identifies information

overload, context fragmentation, and opaque system rationale as the three primary drivers of poor human decision quality in automated supervision tasks [18]. Each panel directly addresses one of these failure modes.

The right Diagnostics panel addresses opacity by presenting the numerical Ccs, the threshold it failed to meet, and a natural-language decomposition of the contributing factors. An example display reads: "Confidence Score: 42% (Threshold: 50%). Contributing factors: (1) Token probability 0.31—model expressed low generative certainty about this step. (2) Semantic alignment 0.49—the proposed action references account identifier ACC-7821, which was not mentioned in the original task specification. Recommend verifying account reference before proceeding." This explanation is generated by the Supervisor Agent as part of its evaluation output, not post-hoc by a separate explanation module.

The operator has three response options: Approve (execute the action as proposed), Reject with Correction (enter a revised action or corrected plan step), or Reject and Terminate (halt the entire task with a reason annotation). Every operator response is logged with a timestamp, operator identifier, response type, and—where a correction is provided—the full text of the corrected plan step. The preference tuple $D = \{x, y_{\text{rejected}}, y_{\text{chosen}}\}$ is populated immediately on response receipt and written to the preference store for the next DPO fine-tuning cycle.

5.6 Direct preference optimisation feedback loop

In the present implementation, each human intervention produces a preference tuple $D = \{x, y_{\text{rejected}}, y_{\text{chosen}}\}$ where x is the full task context at the point of intervention (including the preceding conversation history and the complete chain-of-thought plan), y_{rejected} is the agent's rejected plan step, and y_{chosen} is the human operator's approved or corrected alternative. These tuples are accumulated in a preference store backed by a separate Redis namespace and batched at the end of each epoch of 1,000 tasks for a fine-tuning run on the Supervisor Agent using the Direct Preference Optimisation objective. Fine-tuning is conducted with a learning rate of 1×10^{-5} , a batch size of 16 preference pairs, and a maximum of 3 epochs per fine-tuning cycle; these hyperparameters were set by grid search on the 500-task calibration set.

The practical effect of this mechanism is that the Supervisor progressively learns to pre-emptively route borderline decisions through the Path B self-correction loop—generating structured feedback to the Worker and requesting a revised plan—rather than escalating to the human. This reduces the HITL_BLOCK intervention rate over time without weakening the safety guarantee, because actions are not being approved without review: they are being corrected before reaching the human escalation threshold. The distinction is important and the empirical evidence for the correct interpretation is presented in Sect. 7.3.

6 Experimental setup

6.1 Dataset construction and validation

A dataset of 5000 synthetic enterprise automation tasks was constructed spanning three operationally representative domains. Synthetic data generation was chosen over real enterprise data for three reasons: to avoid the privacy, confidentiality, and data governance obligations that would attend the use of real client data; to enable precise

experimental control over the distribution of task types, difficulty levels, and error modes; and to permit ground-truth labelling of correct versus incorrect task outcomes without reliance on costly or subjective human annotation at scale.

The generation pipeline operated as follows. For each domain, a library of task templates was defined, each specifying the task category, the tool or tools required for execution, the ground-truth correct outcome, and the correctness verification criterion. Concrete task instances were produced by sampling parameter values from domain-specific distributions into these templates. The generation prompts were designed to produce a distribution of task difficulty spanning: straightforward tasks expected to fall in the autonomous execution regime under the Dynamic Intervention Framework (target approximately 60% of the dataset); ambiguous or moderately complex tasks expected to require asynchronous review (target 20%); and genuinely challenging tasks expected to trigger HITL_BLOCK escalation (target 20%). The actual observed distribution was 85.5% autonomous, 0% async review (the async review regime occurred transiently but escalations and autonomous executions dominated the logged routing states), and 14.5% HITL_BLOCK, indicating that the model handled more tasks autonomously than the generation design anticipated—a finding discussed in Sect. 8.

Domain allocations and task subtypes were as follows. Financial Data Processing comprises 1,500 tasks across ten subtypes: transaction categorisation, anomaly flagging, balance reconciliation, conditional transfer initiation, compliance audit, fraud pattern detection, account audit, budget allocation, invoice processing, and tax computation. Code Generation comprises 1500 tasks: SQL query construction, Python script refactoring, API integration code, unit test generation, static security scanning, schema migration scripting, data pipeline construction, microservice scaffolding, bug fixing, and code linting. Customer Support Actions comprises 2000 tasks: refund processing, account permission reset, escalation response drafting, subscription modification, password reset, account merge, billing dispute resolution, data subject access request fulfilment, complaint response drafting, and service level agreement tracking.

Dataset validity was assessed by verifying that the fully autonomous baseline achieved a success rate substantially below 100%—confirming that the dataset contains a realistic proportion of genuinely challenging tasks. The observed 84.3% success rate under the fully autonomous condition provides an implicit validation: a dataset on which an unconstrained capable model achieves 95% or higher would be insufficiently discriminative for evaluating an oversight framework. The complete 5000-task dataset, with all 18 per-task fields documented in the companion data dictionary, is available from the corresponding author.

6.2 Experimental baselines

The Dynamic Intervention Framework was evaluated against two baselines. Baseline A (Fully Autonomous) deployed the Worker Agent without any Supervisor or intervention mechanism. To avoid penalising the baseline with an artificial constraint, the chain-of-thought schema requirement was removed from the Worker's system prompt in the Baseline A condition, allowing the Worker to generate responses directly without the JSON externalisation step. Baseline B (Static Human-in-the-Loop) routed every task decision through human review, simulating the conventional approval gate model at maximum oversight intensity. Human responses for Baseline B were simulated using a

held-out oracle model prompted with access to ground-truth correctness labels, because recruiting human reviewers for 5,000 tasks at evaluation scale was operationally infeasible. Human review latency in Baseline B was modelled as a log-normal distribution with mean 45.6 s and standard deviation 12 s, calibrated on a pilot study involving 200 tasks reviewed by three human evaluators recruited from the authors' professional networks. The pilot reviewers were given the same three-panel interface used in the Dynamic Intervention Framework condition and were asked to review without time pressure, producing a latency distribution representative of thoughtful rather than rushed review.

6.3 Evaluation metrics

Four primary metrics were computed. Task Success Rate was defined as the proportion of tasks for which the final executed outcome exactly matched the ground-truth correct outcome as determined by automated verification: SQL result matching against a reference execution for database tasks, unit test pass/fail for code generation tasks, and semantic entailment scoring against a reference response for customer support drafting tasks. Mean Task Latency was the mean wall-clock time from task submission to final execution completion, inclusive of supervisor evaluation overhead and human review time where applicable. Human Intervention Rate was the proportion of decision steps routed to HITL_BLOCK under the Dynamic Intervention Framework, or equivalently 100% under Baseline B and 0% under Baseline A. False Positive Rate for the Dynamic Intervention Framework condition was the proportion of HITL_BLOCK escalations that the oracle determined were correctly planned actions requiring no correction—a measure of unnecessary interruption that directly quantifies the efficiency cost of over-sensitive routing.

Two secondary metrics were also tracked. Self-Correction Rate was the proportion of Path B retry sequences—decisions that triggered the soft-violation feedback loop—that successfully produced a compliant plan within the $\text{MAX_RETRIES} = 3$ limit without requiring human escalation, indicating the effectiveness of structured supervisor feedback in resolving borderline cases. Epoch Learning Curve was the sequence of per-epoch intervention rates $\text{IR}(e)$ for $e \in \{1, 2, 3, 4, 5\}$, tracking how the Direct Preference Optimisation feedback mechanism reduces the human oversight burden over time. Statistical significance for binary success rate comparisons between conditions was assessed using the chi-squared test at the $\alpha = 0.05$ significance level.

6.4 Implementation environment

All experiments were conducted on a cloud computing instance provisioned with 8 NVIDIA A100 GPUs (40 GB memory each), 128 GB of DDR4 RAM, and a 2 TB NVMe SSD. The Worker Agent (GPT-4o) was accessed via the OpenAI application programming interface. The Supervisor Agent (Llama 3 8B fine-tuned) was deployed locally using the Hugging Face Transformers library version 4.41 with bitsandbytes 8-bit quantisation. The embedding model (text-embedding-3-large) was accessed via the OpenAI embeddings endpoint. Redis 7.2 served as both the in-memory state manager and the vector index backend, using the RediSearch module for cosine similarity queries. The Open Policy Agent version 0.64 was deployed as a sidecar Docker container, receiving plan step evaluations via HTTP at the `/v1/data/policy/allow` endpoint. The human operator console was implemented as a Next.js application with a persistent WebSocket

Table 2 Overall system performance comparison

Metric	Baseline A (fully autonomous)	Baseline B (static HITL)	Proposed DIF
Task success rate	84.3%	99.1%	98.2%
Mean task latency (s)	1.2	45.6	4.8
Human intervention rate	0%	100%	14.5%
False positive rate (DIF)	N/A	N/A	3.2%
Self-correction rate (Path B)	N/A	N/A	76.4%

Table 3 Domain-wise task execution and post-intervention accuracy

Domain	Total tasks	Auto-executed	HITL_BLOCK	Success rate (Post-Intervention)
Financial	1500	1,050 (70%)	450 (30%)	99.5%
Code Generation	1500	1,230 (82%)	270 (18%)	97.8%
Customer Support	2000	1,960 (98%)	40 (2%)	98.6%

connection managed by Socket.IO for real-time state synchronisation. System orchestration—the overall agent loop, branching logic, and state transitions — was managed using LangGraph 0.2.

7 Results and analysis

7.1 Overall system performance

The results in Table 2 confirm the principal hypothesis of this work. The Dynamic Intervention Framework achieves a task success rate of 98.2%, compared to 84.3% under the fully autonomous baseline and 99.1% under the static Human-in-the-Loop baseline. The 0.9 percentage-point gap between the Dynamic Intervention Framework and Baseline B is not statistically significant (chi-squared test, $\chi^2 = 1.89$, $p = 0.17$), confirming that the proposed framework achieves safety performance statistically equivalent to full human oversight. The 13.9 percentage-point improvement over Baseline A is highly significant ($\chi^2 = 314.7$, $p < 0.001$), demonstrating that the supervised architecture meaningfully reduces error rates relative to unchecked autonomy.

The mean task latency of 4.8 s for the Dynamic Intervention Framework represents a 9.5-fold reduction relative to Baseline B (45.6 s), driven primarily by the elimination of human review from 85.5% of task decision steps. The per-step overhead introduced by the Supervisor evaluation—approximately 0.3 s for the Ccs computation and policy check—is modest relative to the latency savings from avoided human review and remains within the real-time operational requirement established in Sect. 3. The false positive rate of 3.2% indicates that the routing logic occasionally escalates safe actions to human review unnecessarily. While small in proportional terms, at a deployment scale of 10,000 decisions per hour this would generate approximately 320 unnecessary operator interruptions per hour, identifying false positive reduction as an important target for future optimisation. The self-correction rate of 76.4% for Path B retry sequences demonstrates that the Worker Agent, when provided with structured feedback from the Supervisor, successfully produces a compliant plan within the retry limit in roughly three-quarters of borderline cases, significantly reducing the fraction of such cases that ultimately escalate to human intervention (Table 3).

7.2 Domain-specific performance

The domain-stratified results reveal meaningful variation in how the confidence-based routing distributes oversight load across task types, confirming that the Contextual

Table 4 Intervention rate across training epochs

Epoch	Tasks processed	Interventions triggered	Intervention rate
Epoch 1 (Start)	1–1000	210	21.0%
Epoch 2	1001–2000	165	16.5%
Epoch 3	2001–3000	142	14.2%
Epoch 4	3001–4000	128	12.8%
Epoch 5 (End)	4001–5000	115	11.5%

Confidence Score adapts appropriately to genuine risk heterogeneity rather than applying a uniform intervention criterion. Financial data processing generated the highest HITL_BLOCK intervention rate (30%), reflecting two domain-specific factors: the policy-as-code rules imposed the strictest per-step constraints in this domain—autonomous approval limits, mandatory field completeness rules for transaction records, and regulatory compliance checks—and the mean semantic similarity between individual financial transaction steps and the original high-level goal description was lower than in the other two domains, reflecting the technical specificity of financial action vocabulary.

Code generation produced an intermediate intervention rate of 18%, consistent with the greater surface area for subtle logical errors in programmatic outputs and with the observation that SQL and code actions exhibit more token-level uncertainty on novel query patterns than on common code idioms. The Supervisor's policy-as-code rules for code generation focused primarily on ungarded database operations and security-sensitive API calls, a narrower scope than the financial domain's rules. Customer support actions triggered HITL_BLOCK escalation for only 2% of tasks, reflecting the comparatively lower consequence severity of support actions, the close semantic alignment between natural-language support actions and their goal descriptions, and the absence of hard-coded value limits in the customer support policy-as-code rules.

Post-intervention accuracy was highest in the Financial domain at 99.5%, validating that the HITL_BLOCK escalations in that domain were genuinely warranted and that human correction consistently produced correct outcomes. The 97.8% post-intervention accuracy in the Code Generation domain is the lowest of the three, attributable on inspection to a subset of complex multi-file refactoring tasks in which the oracle correction itself was suboptimal—the task specification was ambiguous with respect to the scope of the refactoring, and even access to ground-truth labels did not fully resolve the ambiguity. This class of failure motivates the co-editing interface research direction identified in Sect. 8.

7.3 Learning effect of the feedback loop

Table 4 tracks the per-epoch intervention rate across the five 1,000-task training sequences. The rate declined monotonically from 21.0% in Epoch 1 to 11.5% in Epoch 5, a relative reduction of 45.2% over the course of the experiment. The absolute reduction is greatest between Epochs 1 and 2 (4.5 percentage points), consistent with the hypothesis that early interventions expose the most common and learnable error patterns—situations in which the Supervisor's pre-DPO judgment was consistently overriding safe actions that the oracle consistently approved—and that the marginal returns to further fine-tuning diminish as the preference dataset covers progressively rarer edge cases.

A critical interpretive point is that the monotonic decline in intervention rate was not accompanied by any statistically detectable degradation in task success rate, which

remained stable between 97.9 and 98.5% across the five epochs (within-epoch variation, not separately tabled). This stability is the key evidence that the Direct Preference Optimisation mechanism is achieving the intended effect: teaching the Supervisor to preempt errors through Path B self-correction rather than simply becoming less sensitive to risky actions. A Supervisor that reduced its intervention rate by lowering its confidence thresholds would produce a similar IR trajectory in Table 4, but would be expected to show a corresponding degradation in success rate as more genuinely dangerous actions passed through the autonomous gate without detection. The absence of such degradation provides strong evidence against this alternative explanation. The two effects—declining IR and stable success rate—together constitute the primary validation of the feedback loop's design.

8 Discussion

8.1 Synthesis and interpretation of findings

The central empirical finding of this study—that a confidence-based adaptive routing architecture can achieve safety performance statistically equivalent to full human oversight while reducing the required volume of human review by 85.5%—directly addresses the oversight paradox identified in Sect. 1. The finding does not imply that human oversight is unimportant: the 99.5% post-intervention accuracy in the Financial domain confirms that the 30% of Financial tasks that do reach the human operator genuinely benefit from human judgment and that the corrections provided are consistently valuable. What the finding implies is that the value of human oversight is concentrated in a specific, identifiable minority of decisions, and that a system capable of accurately identifying that minority can realise the full efficiency benefit of autonomy while preserving the safety benefit of oversight for the cases that actually require it.

The domain-stratified analysis extends this conclusion in an important direction. The variation in intervention rates across domains—30% for Financial, 18% for Code Generation, 2% for Customer Support—reflects genuine differences in the risk profiles and semantic characteristics of the three task types, not noise or miscalibration. A static Human-in-the-Loop architecture that applied uniform oversight to all three domains would either over-supervise Customer Support tasks (2% genuine escalation rate, yet 100% oversight required) or under-supervise Financial tasks (30% genuine escalation rate, yet only a fixed fraction reviewed). The Dynamic Intervention Framework's adaptive routing automatically assigns oversight intensity proportional to the observed risk level of each domain, without requiring domain-specific rules to be manually specified for every task type. This adaptivity is one of the most practically valuable properties of the framework from a deployment standpoint.

The learning effect documented in Table 4 raises a question about the asymptotic behaviour of the intervention rate under continued operation beyond the 5000-task experimental window. The slowing rate of decline across epochs—4.5 percentage points between Epochs 1 and 2, 1.3 percentage points between Epochs 4 and 5—is consistent with a model approaching a floor determined by the irreducible proportion of genuinely novel, high-risk, or inherently ambiguous decisions that no amount of historical training data can allow the Supervisor to handle autonomously. Estimating this floor theoretically, and providing practitioners with a reliable forecast of the long-term steady-state

intervention rate for a given deployment context, is identified as an open research question in Sect. 8.3.

8.2 Practical managerial significance

Beyond its technical contributions, this work has direct practical implications for managers and executives responsible for agentic artificial intelligence deployments. Four dimensions of managerial significance are addressed: economic reallocation, regulatory compliance, operator trust, and long-term cost trajectory.

From an economic perspective, an organisation currently operating a static Human-in-the-Loop model—in which trained reviewers spend working hours approving agent decision steps—can expect the Dynamic Intervention Framework to reallocate approximately 85% of that review effort to higher-value activities. Depending on the compensation rate and seniority level of the reviewers involved, this can represent a material reduction in ongoing operational expenditure. The remaining 14.5% of decisions that require human review are, by construction, the decisions that most benefit from it: the high post-intervention accuracy rates across all three domains confirm that human review of escalated decisions consistently improves outcomes, meaning that the review hours retained are invested where they generate the most value.

From a regulatory compliance perspective, the framework's audit trail architecture provides a natural compliance artefact. Every routing decision is logged with its Ccs value, the threshold against which it was evaluated, and—for HITL_BLOCK events—the operator identity, timestamp, and disposition. Because the full task state is serialised at the point of escalation, any escalated decision can be exactly reproduced and audited after the fact, supporting post-hoc regulatory review without requiring real-time human monitoring of every agent action. For regulated industries—financial services under MiFID II and Dodd-Frank, healthcare under HIPAA, legal services under bar association supervision requirements—this audit trail may be sufficient to demonstrate that autonomous actions were taken within a governed oversight framework, potentially enabling regulatory approval for a broader scope of autonomous operation than would be possible under unstructured deployment.

8.3 Open research questions

The findings of this study suggest five structured directions for future investigation, presented as Open Research Questions to provide an explicit agenda for subsequent work.

1. ORQ-1 (Adversarial Robustness). The Contextual Confidence Score is computed from features—token log-probabilities and embedding-space distances—that a sophisticated adversarial prompt could potentially manipulate. A prompt engineered to produce high token-level generation probability for a dangerous action, by exploiting statistical regularities in the training distribution, could artificially inflate Ccs and cause the framework to route a harmful action through the autonomous gate. Evaluating the robustness of the framework against such confidence-inflation attacks—analogueous to adversarial examples in computer vision, adapted to the continuous text generation setting—is a critical prerequisite for deployment in environments where adversarial manipulation is plausible, including cybersecurity operations, financial fraud detection, and politically sensitive content moderation.

2. ORQ-2 (Multi-Modal Extension). The present framework processes textual inputs exclusively. Many high-stakes enterprise workflows involve non-textual inputs—scanned financial documents, medical images, engineering schematics, audio recordings—whose semantic content is not captured by text embeddings. Extending the Ccs metric to incorporate multi-modal uncertainty signals requires developing analogues of both the Plogits and Ssemantic components for non-textual modalities. This involves both the technical challenge of multi-modal uncertainty estimation and the theoretical challenge of defining a meaningful semantic alignment measure between, for example, a proposed action described in text and a goal partially specified through an image.
3. ORQ-3 (Asymptotic Intervention Rate and Learning Curve Modelling). The declining intervention rate trajectory observed in Table 4 suggests approach to an asymptotic floor, but the experimental window of 5000 tasks is insufficient to characterise that floor empirically. Theoretically characterising the asymptote—what determines its value, how it depends on domain characteristics and task distribution, and how it can be estimated in advance without running the full experimental sequence—would be of direct practical value for deployment planning. A Bayesian model of the learning curve, fitted to per-epoch data and used to produce a predictive posterior over future intervention rates, is a promising methodological approach.
4. ORQ-4 (Cross-Domain Threshold Transfer). The thresholds $T_{high}=0.85$ and $T_{low}=0.50$ were calibrated on the three specific task domains evaluated in this study. Whether these thresholds transfer to new deployment domains without recalibration—or whether domain-specific calibration is always required—has significant implications for the cost and time required to deploy the framework in new operational contexts. A transfer study examining how calibration performance degrades when thresholds calibrated on one domain are applied to a different domain, and identifying domain features that predict transferability, would help practitioners decide when recalibration is necessary.
5. ORQ-5 (Human Factors in Decision Interface Design). The three-panel decision console was designed on human–computer interaction principles, but its effect on operator decision quality, cognitive fatigue under sustained use, and accuracy under time pressure has not been formally evaluated. Prior research on decision support systems suggests that interface design choices can materially affect both the quality of individual decisions and the rate of operator burnout over extended shift periods [18]. A controlled human factors study comparing the Dynamic Intervention Framework interface against alternative designs—including a minimal text-only interface and a current industry-standard email or chat notification approach—would provide an evidence base for interface design recommendations applicable to the broader category of human-AI oversight interfaces.

8.4 Limitations

Several limitations of the present work require explicit acknowledgement. The most significant is the reliance on synthetic evaluation data. The generation pipeline was designed to produce a realistic distribution of task types and difficulty levels, but synthetic tasks inevitably differ from real enterprise tasks in ways that cannot be fully controlled. Real tasks frequently involve ambiguous or contradictory instructions, implicit

domain knowledge that is absent from the task specification, partial information that must be resolved through clarification, and temporal dependencies on prior workflow state that are difficult to capture in isolated task instances. Whether the Dynamic Intervention Framework's performance generalises from the controlled synthetic environment to the messy reality of live enterprise deployment is the most important open empirical question, and validation on real enterprise data with real operators is the primary priority for future work.

A second limitation is the use of oracle simulation for Baseline B human responses. The oracle model provides a reproducible and controlled approximation of human review, but it does not capture the cognitive phenomena that characterise real human oversight at scale: attention fatigue from sustained review sessions, expertise variability across reviewers with different backgrounds, cognitive biases that affect decision consistency, and the tendency of operators under time pressure to approve rather than carefully evaluate borderline cases. These phenomena collectively mean that the 99.1% success rate attributed to Baseline B may overestimate the achievable performance of static Human-in-the-Loop oversight in practice, which would make the Dynamic Intervention Framework's 98.2% rate an even more favourable comparison.

9 Conclusion

This paper has proposed and evaluated the Dynamic Intervention Framework, an architecture for agentic artificial intelligence that resolves the oversight paradox at the heart of enterprise autonomous system deployment. By treating human intervention as a dynamically allocated resource governed by a continuously evaluated Contextual Confidence Score, the framework achieves three goals simultaneously: it maintains task accuracy statistically equivalent to full human supervision; it eliminates the need for human review from 85.5% of decision steps, preserving the efficiency rationale for agentic deployment; and it progressively reduces the remaining oversight burden through a Direct Preference Optimisation feedback loop that transfers human safety judgment into the Supervisor's model.

The empirical evaluation on 5000 synthetic enterprise tasks across three operationally representative domains confirmed that the proposed architecture delivers on all three objectives. The domain-stratified analysis demonstrated that the confidence-based routing correctly reflects the genuine risk heterogeneity of different task types, assigning proportional oversight intensity in a way that static checkpoint models cannot. The longitudinal analysis of the feedback loop provided evidence that the learning mechanism is achieving its intended effect—teaching the Supervisor to pre-empt errors rather than simply becoming less sensitive to risk—through the key empirical signature of declining intervention rate accompanied by stable success rate.

The work is bounded by its reliance on synthetic data and simulated human responses, and these limitations define the primary agenda for future investigation: real-world validation with live enterprise data and operators, adversarial robustness evaluation, multi-modal extension, and characterisation of the asymptotic learning floor. The five open research questions identified in Sect. 8.3 provide a structured agenda for this investigation. The practical managerial implications discussed in Sect. 8.2—cost reallocation, regulatory compliance artefacts, trust calibration, and learning-adjusted deployment cost forecasting—indicate that the framework has near-term practical value for organisations

navigating the transition to enterprise agentic operations, and that the governance challenge it addresses will only become more pressing as the capability and scope of agentic deployments continue to expand.

Acknowledgements

Generative artificial intelligence tools were used solely for grammar and language editing in the preparation of this manuscript. All research design, experimental methodology, quantitative analysis, interpretation of results, and intellectual contributions are the sole work of the listed authors, in full compliance with the journal's editorial policies on the use of generative AI.

Author contribution

Madhvesh Kumar and Deepika Singh conceptualized the Dynamic Intervention Framework (DIF) and designed the multi-agent "Supervisor-Worker" architecture. Madhvesh Kumar conducted the experiments, managed the dataset of 5000 enterprise automation tasks, and performed the quantitative performance analysis. Deepika Singh formulated the Contextual Confidence Score metric, implemented the reinforcement learning (RLHF) loop, and prepared the figures and tables. Both Madhvesh Kumar and Deepika Singh wrote the main manuscript text. All authors reviewed and approved the final manuscript.

Funding

No funding was received for this research.

Data availability

The 5000-task synthetic enterprise dataset described in Sect. 6.1 is available from the corresponding author on reasonable request. An 18-column per-task data dictionary is provided in the companion spreadsheet.

Declarations

Ethical approval and consent to participate

Not applicable. This study used only synthetic data and did not involve human participants in data collection. Ethical approval was therefore not required. Not applicable. This study did not involve human participants.

Consent for publication

Not applicable. This study did not involve human participants, and all data used are synthetic.

Competing interest

The authors declare no competing interests.

Received: 24 February 2026 / Accepted: 27 April 2026

Published online: 28 May 2026

References

1. Wang L, Ma C, Feng X, et al. A survey on large language model based autonomous agents. *Front Comput Sci*. 2024;18(6):186345. <https://doi.org/10.1007/s11704-024-40231-1>.
2. Orq.ai. Why multi-agent LLM systems fail: key issues explained. Orq.ai Blog. 2025. <https://orq.ai/blog/multi-agent-llm-failur-e-modes> [Web publication; no persistent available]. Accessed Jan 2026
3. SiliconANGLE Media. Human-in-the-loop has hit the wall — it is time for AI to oversee AI. SiliconANGLE. 2026. <https://siliconangle.com> [Web publication]. Accessed Jan 2026
4. Page MJ, McKenzie JE, Bossuyt PM, et al. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*. 2021;372:n71. <https://doi.org/10.1136/bmj.n71>.
5. Russell S, Norvig P. *Artificial Intelligence: A Modern Approach*. 4th ed. Hoboken: Pearson; 2020. ISBN 978-0-13-468599-1.
6. Li Y, Zhang Y, Sun L. MetaAgents: simulating interactions of human behaviors for LLM-based task-oriented coordination. *arXiv*. 2023. <https://doi.org/10.48550/arXiv.2310.06500>.
7. Wu Q, Bansal G, Zhang J, et al. AutoGen: enabling next-gen LLM applications via multi-agent conversation. *arXiv [Preprint]*. 2023. <https://doi.org/10.48550/arXiv.2308.08155>
8. Hong S, Zheng X, Chen J, et al. MetaGPT: meta programming for a multi-agent collaborative framework. *Proceedings of ICLR*. 2024. <https://doi.org/10.48550/arXiv.2308.00352>.
9. Huang J, Chen X, Mishra S, et al. Large language models cannot self-correct reasoning yet. *Proceedings of ICLR*. 2024. <https://doi.org/10.48550/arXiv.2310.01848>.
10. Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback. *Adv Neural Inf Proc Syst*. 2022;35:27730–44. <https://doi.org/10.48550/arXiv.2203.02155>.
11. Kadavath S, Conerly T, Askell A, et al. Language models (mostly) know what they know. *arXiv [Preprint]*. 2022. <https://doi.org/10.48550/arXiv.2207.05221>
12. Kuhn L, Gal Y, Farquhar S. Semantic uncertainty: linguistic invariances for uncertainty estimation in natural language generation. *Proc ICLR*. 2023. <https://doi.org/10.48550/arXiv.2302.09664>.
13. Ruan Y, Dong H, Wang A, et al. Identifying the risks of LM agents with an LM-emulated sandbox. *Proc ICLR*. 2024. <https://doi.org/10.48550/arXiv.2309.15817>.
14. Bai Y, Jones A, Ndousse K, et al. Constitutional AI: harmlessness from AI feedback. *arXiv [Preprint]*. 2022. <https://doi.org/10.48550/arXiv.2212.08073>
15. National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. Gaithersburg: NIST; 2023. <https://doi.org/10.6028/NIST.AI.100-1>

16. Guo C, Pleiss G, Sun Y, Weinberger KQ. On calibration of modern neural networks. Proc ICML. 2017. <https://doi.org/10.48550/arXiv.1706.04599>.
17. Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models. *Adva Neural Inf Process Syst*. 2022;35:24824–37. <https://doi.org/10.48550/arXiv.2201.11903>.
18. Lee JD, See KA. Trust in automation: designing for appropriate reliance. *Hum Factors*. 2004;46(1):50–80. https://doi.org/10.1518/hfes.46.1.50_30392.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.